Work Effectively in the GitHub Copilot App

Session 20 | Product and Delivery Teams | Beginner

The app is the classroom

This track does not use Copilot as a background writing tool.

Learners work in the GitHub Copilot app with repository context and reviewable artifacts.

Delivery Decision Studio is the six-session path. **Service Request Portal** is the initiative.

The app is a workspace, not an answer engine

The chat is one place to work. Projects, source references, issues, and saved artifacts make the work reviewable after the conversation ends.

```
Gather evidence → form a proposal → review it → save the decision → verify it
```

This separates the model's narrative from facts the team can check.

Context has a cost

Context choice	When it helps
One named file	The answer depends on a specific rule or API
A project or repository	The task needs relationships across files
An issue or pull request	The task needs delivery history and discussion
No repository context	The work is general and contains no project facts

Tell Copilot why each source matters. **Do not make it guess.**

Separate facts, assumptions, and decisions

Type	Example
Fact	"Issue 42 names a response-time problem."
Assumption	"The mobile client uses the documented endpoint."
Decision	"The product owner will confirm the client behavior."

This makes uncertainty visible. A polished paragraph can hide all three.

Know the main work areas

Area	Use it for
Chats	Exploration and interviews
Projects	Repository-grounded work
My work	Issues, pull requests, and sessions
Search	Finding repository information
Automations	Recurring or on-demand tasks
Customize	Skills, agents, MCP servers, and canvases

Context comes before the prompt

Give Copilot the smallest approved context that can answer the question.

Name the files, issues, or repository that should support the answer.

Choose the session mode

Mode	Use in this path
Interactive	Questions, interviews, and reviewed drafts
Plan	Review a multi-step approach before changes
Autopilot	Bounded work with approved autonomy

Session 20 uses Interactive, then Plan. The learner reviews every write.

Use verified context controls

- `@` adds a file.
- `#` references an issue.
- `/` opens commands.
- `/context` shows current context usage.
- `/compact` summarizes a long active session.
- `/clear` or `/reset` starts fresh.

Ask for sources

```
Explain the initiative for a product owner.
Cite the supplied source for each factual claim.
Mark missing information as unknown.
```

A confident answer without a source is still unverified.

Inspect what Copilot can use

- Skills provide repeatable procedures.
- Agents carry role-specific instructions and tools.
- MCP servers connect Copilot to systems.
- Plugins install reusable packages that can include MCP servers and other customizations.
- Canvases provide visible shared state.
- Automations repeat bounded work.

Availability depends on the plan, policy, and installation.

Use one repeatable work pattern

```
Read -> propose -> review -> write -> verify
```

Every later Module 7 session uses this pattern.

Conversation should become an artifact

The lab creates `initiative-context-brief.md` with:

- facts and sources;
- unknowns;
- useful GitHub artifacts;
- the next decision and owner.

The artifact must make sense without the chat history.

Evidence changes the brief

```
Initial source
  → first brief
  → approved late evidence
  → focused revision
  → fresh-session verification
```

The brief works only while it matches approved evidence.

Lab

1. Open the repository as a Copilot project.
2. Inspect the available capabilities.
3. Compare an unscoped answer with a grounded answer.
4. Create and commit the initiative context brief.
5. Apply approved late evidence.
6. Review and commit the focused revision.
7. Ask `/rubber-duck` for independent critique.
8. Verify the brief in a fresh session.

Stop if the app or repository context is unavailable.